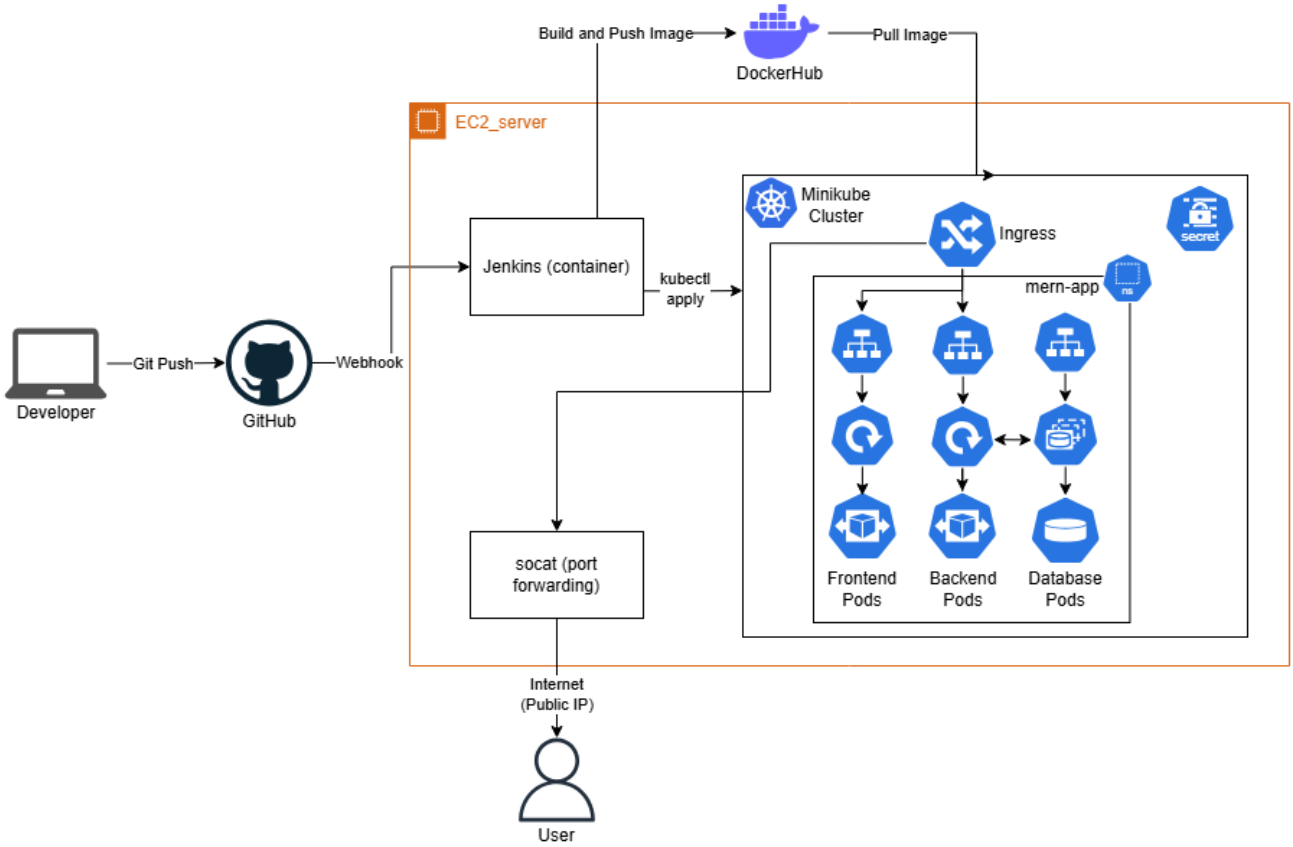
MERN STACK DEPLOYMENT GUIDE

AWS EC2 • Docker • Jenkins CI/CD • Kubernetes

Step-by-Step Deployment from AWS Console to Production

Architecture Overview

This guide deploys a full-stack MERN (MongoDB, Express, React, Node.js) application on AWS EC2 using a Jenkins CI/CD pipeline that builds Docker images and deploys them to a local Minikube Kubernetes cluster running on the same EC2 instance. Traffic is forwarded from the EC2 public IP to the Minikube ingress via socat.



Repository File Structure

File / Path	Purpose
`Jenkinsfile`	Declarative CI/CD pipeline: install deps, lint, build React, Docker build/push, deploy to K8s
`docker-compose.yml`	Local development stack (frontend, backend, MongoDB)
`backend/Dockerfile`	Node.js 20 Alpine image, runs as non-root user, exposes port 5000
`backend/.env ,/.env.example`	Environment variables: PORT, MONGO_URI, JWT_SECRET
`backend/.dockerignore`	Excludes node_modules, env, logs from Docker build context
`frontend/Dockerfile`	Multi-stage: builds React app with Node then serves via nginx:alpine
`frontend/nginx.conf`	Serves React SPA, proxies /api/* to backend service on port 5000
`frontend/.dockerignore`	Excludes node_modules, build/dist artifacts, env files
`K8s/namespace.yaml`	Creates mern-app Kubernetes namespace
`K8s/secrets.yaml`	Stores MONGO_URI and JWT_SECRET as K8s Opaque Secret
`K8s/mongo/statefulset.yaml`	MongoDB StatefulSet (mongo:7), mounts PVC for data persistence
`K8s/mongo/pvc.yaml`	2Gi PersistentVolumeClaim for MongoDB data
`K8s/mongo/service.yaml`	Headless ClusterIP service for MongoDB on port 27017
`K8s/server/deployment.yaml`	Backend deployment (2 replicas), readiness & liveness probes on /api/health
`K8s/server/service.yaml`	ClusterIP service for backend on port 5000
`K8s/server/hpa.yaml`	HPA: 2–10 replicas, scales on CPU >70% or Memory >80%
`K8s/client/deployment.yaml`	Frontend deployment (2 replicas), serves nginx on port 80
`K8s/client/service.yaml`	ClusterIP service for frontend on port 80
`K8s/client/hpa.yaml`	HPA: 2–6 replicas, scales on CPU >70%
`K8s/client/ingress.yaml`	Nginx Ingress: routes /api → server-service:5000, / → client-service:80

STEP 1 Launch an EC2 Instance (AWS Console)

Log in to the AWS Management Console and navigate to EC2 → Instances → Launch Instance.

1.1 Choose AMI and Instance Type

- AMI: Amazon Linux 2023 (or Amazon Linux 2) - 64-bit (x86)
- Instance type: m7i-flex.large (2 vCPUs, 8 GB RAM for Minikube)

1.2 Configure Key Pair

- Create a new key pair (RSA, .pem format) or select an existing one
- Download and save the .pem file - you cannot re-download it

1.3 Configure Security Group (Inbound Rules)

Type	Protocol	Port Range	Source / Purpose
SSH	TCP	22	Your IP - for SSH access
Custom TCP	TCP	8080	Your IP - Jenkins Web UI
HTTP	TCP	80	0.0.0.0/0 - Application access via public IP

1.4 Storage

- Set root volume to at least 20 GB (gp3) - Minikube and Docker images need space

1.5 Launch

- Review settings and click Launch Instance
- Note the Public IPv4 address from the instance details panel - you will use it throughout

NOTE Wait 2–3 minutes for the instance to fully initialize before connecting via SSH.

STEP 2 Connect to EC2 via SSH

From your local terminal, set the correct permissions on the key file, then SSH into the instance:

```
# Set permissions on your downloaded key pair
chmod 400 your-key.pem

# SSH into the EC2 instance (replace <EC2_PUBLIC_IP> with your instance IP)
ssh -i your-key.pem ec2-user@<EC2_PUBLIC_IP>
```

STEP 3 Install and Start Docker

Run the following commands on the EC2 instance to install Docker and add your user to the docker group:

```
# Install Docker
sudo yum install docker -y

# Add ec2-user to the docker group (avoids needing sudo for docker commands)
sudo usermod -aG docker $USER

# Activate the new group membership in the current session
newgrp docker

# Start Docker and enable it to start on boot
sudo systemctl start docker
sudo systemctl enable docker
```

TIP Run 'docker ps' to verify Docker is running. You should see an empty container list with no errors.

STEP 4 Install kubectl and Minikube

kubectl is the Kubernetes CLI and Minikube provides a single-node local cluster on the EC2 instance:

```
# Download the latest stable kubectl binary
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"

# Make it executable and move to PATH
chmod +x kubectl
sudo mv kubectl /usr/local/bin/

# Download Minikube
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64

# Install Minikube and clean up
sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64
```

STEP 5 Start Minikube and Configure Ingress

Start Minikube using the Docker driver, flatten the kubeconfig for Jenkins to use, then enable the nginx Ingress controller:

```
# Start Minikube cluster using Docker as the driver
minikube start --driver=docker

# Flatten kubeconfig into a single portable file (used by Jenkins later)
kubectl config view --minify --flatten > ~/.kube/config-flat

# Enable the nginx Ingress addon (required for K8s/client/ingress.yaml)
minikube addons enable ingress

# Install socat - required to forward EC2 public port 80 to Minikube
sudo yum install -y socat
```

WHY SOCAT? Minikube runs inside Docker and has its own internal IP. socat forwards traffic arriving at EC2 port 80 to the Minikube cluster IP so the app is reachable via the EC2 public IP.

STEP 6 Launch Jenkins in Docker

Run Jenkins as a Docker container, mounting the Docker socket and the Minikube kubeconfig so Jenkins pipelines can build Docker images and deploy to the Kubernetes cluster:

```
# Capture the host docker group ID (Jenkins container needs it to call docker)
DOCKER_GID=$(getent group docker | cut -d: -f3)
```

```
# Run Jenkins container
docker run -d \
  --name jenkins \
  -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v /home/ec2-user/.kube:/home/ec2-user/.kube:ro \
  -v /usr/local/bin/kubectl:/usr/local/bin/kubectl \
  --group-add $DOCKER_GID \
  --restart=always \
  jenkins/jenkins:lts
```

VOLUMES	docker.sock lets Jenkins build/push images. The .kube mount provides the kubeconfig so the pipeline can run kubectl commands against Minikube.

6.1 Install Docker inside the Jenkins container

The Jenkins container needs its own Docker client binary to run docker commands in pipeline stages:

```
docker exec -it --user root jenkins bash -c "apt-get update && apt-get install -y docker.io"
```

6.2 Fix permissions

```
# Allow the Jenkins container to use the host Docker socket
sudo chmod 666 /var/run/docker.sock

# Make the flattened kubeconfig readable
sudo chmod 644 /home/ec2-user/.kube/config
```

6.3 Connect Jenkins to the Minikube Docker network

Jenkins needs to reach Minikube's internal IP. Connect the Jenkins container to the minikube Docker network:

```
docker network connect minikube jenkins
```

6.4 Retrieve the initial admin password

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

SAVE THIS	Copy the password printed above - you will paste it into the Jenkins setup wizard in the next step.

STEP 7 Jenkins Initial Setup (Web UI)

Open a browser and navigate to your Jenkins instance:

```
http://<EC2_PUBLIC_IP>:8080
```

7.1 Unlock Jenkins

- Paste the initial admin password retrieved in Step 6.4
- Click "Continue"

7.2 Install Plugins

- Select "Install suggested plugins" and wait for installation to complete
- After completion, additionally install: GitHub Integration, Docker Pipeline and NodeJS Plugins

7.3 Create Admin User

- Fill in username, password, full name, and email
- Click "Save and Continue" then "Save and Finish"

STEP 8 Configure Jenkins Tools and Credentials

8.1 Configure NodeJS Tool

The Jenkinsfile declares tools { nodejs 'NodeJS 24.14.1' }. Register this tool name in Jenkins:

- Navigate to: Manage Jenkins → Tools → NodeJS installations
- Click "Add NodeJS"
- Set Name to: NodeJS 24.14.1 (must match exactly)
- Check Install automatically, select the matching version
- Click "Save"

8.2 Creating Personal Access Tokens (PAT)

GitHub PAT

- Go to **GitHub** → **Settings** (top-right avatar) → **Developer settings** (bottom of left sidebar) → **Personal access tokens** → **Tokens (classic)**
- Click **"Generate new token (classic)"**
- Give it a note like jenkins-mern-app
- Set expiration (90 days recommended)
- Select these scopes:
  - repo (full repo access - needed to clone private repos)
  - admin:repo_hook (needed if Jenkins auto-manages webhooks)
- Click **"Generate token"** and **copy it immediately** — GitHub won't show it again

Docker Hub PAT

- Go to **DockerHub** → **Account Settings** (top-right avatar) → **Personal access tokens** → **Generate new token**
- Give it a description like jenkins-mern-deploy
- Set Access permissions to **Read, Write, Delete**
- Click **"Generate"** and **copy the token immediately**

8.3 Add Credentials

Go to Dashboard → Manage Jenkins → Credentials → System → Global credentials → Add Credentials.

GitHub PAT

- Kind: Username with password
- Username: your GitHub username
- Password: your PAT token
- ID: github-credentials

Docker Hub

- Kind: Username with password
- Username: your Docker Hub username

- Password: your Docker Hub password or access token
- ID: dockerhub-credentials

MongoDB URI

- Kind: Secret text
- Secret: your MongoDB connection string
- ID: mongo-uri

GitHub Token

- Kind: Secret text
- Secret: your github pat token
- ID: github-token

8.24 Connect Jenkins to Github

Go to Dashboard → Manage Jenkins → System, scroll to the GitHub section:

- Click Add GitHub Server
- Name: GitHub
- Set API URL to: https://api.github.com
- Select github-token
- Click Test connection — should say Credentials verified
- Click “Save”

STEP 9 Create the Jenkins Pipeline Job

9.1 New Pipeline Item

- From the Jenkins dashboard, click "New Item"
- Enter a name, e.g., mern-app-pipeline
- Select "Pipeline" and click OK

9.2 Configure Webhook for Auto-Trigger

- In GitHub → repository Settings → Webhooks → Add webhook
- Payload URL: http://<EC2_PUBLIC_IP>:8080/github-webhook/
- Content type: application/json
- Select "Just the push event" and save it
- In Jenkins: enable Build Triggers → GitHub hook trigger for GITScm polling

9.3 Configure Pipeline Source

- Under "Pipeline" section, set Definition to: Pipeline script from SCM
- SCM: Git
- Repository URL: your GitHub repository URL
- Add credentials if the repo is private
- Branch: */main (or your default branch)
- Script Path: Jenkinsfile
- Click "Save"

STEP 10 Understand the CI/CD Pipeline (Jenkinsfile)

The Jenkinsfile defines the following stages executed in order:

Stage	What It Does
`Checkout`	Clones the Git repository using the configured SCM credentials
`Install Dependencies`	Runs npm ci in parallel for both frontend/ and backend/ directories
`Lint Frontend`	Runs ESLint with a max-warnings 100 threshold to catch code quality issues
`Build React App`	Runs npm run build inside frontend/ to produce the production dist/
`Build Docker Images`	Builds frontend and backend Docker images tagged with $IMAGE_NAME-frontend:$BUILD_NUMBER and backend equivalent
`Push to Docker Hub`	Logs in to Docker Hub using the stored credentials and pushes both tagged images
`Setup Kubeconfig`	Copies the flattened kubeconfig from the host into /var/jenkins_home/.kube/config inside the Jenkins container
`Deploy to Kubernetes`	Updates image tags in K8s YAML with sed, then kubectl apply all manifests; waits for rollout of server and client deployments

STEP 11 Trigger and Monitor the Pipeline

Trigger the pipeline manually for the first run, or push a commit to the GitHub repo if webhooks are configured:

11.1 Manual Trigger

- From the Jenkins dashboard, click on your pipeline job then click "Build Now"
- Click on the build number in Build History to open the build details
- Click "Console Output" to watch the pipeline run in real time

11.2 Expected Pipeline Output

```
[Pipeline] stage: Checkout
Cloning repository...
[Pipeline] stage: Install Dependencies
...
[Pipeline] stage: Build Docker Images
Successfully built <IMAGE_ID>
[Pipeline] stage: Push to Docker Hub
Pushing nikhilesh2701/mern-app-frontend:<BUILD_NUMBER>
[Pipeline] stage: Deploy to Kubernetes
namespace/mern-app configured
deployment.apps/server configured
deployment.apps/client configured
Waiting for deployment 'server' rollout to finish...
deployment 'server' successfully rolled out
Pipeline completed successfully!
```

STEP 12 Verify Kubernetes Deployments

After the pipeline succeeds, confirm all Kubernetes resources are running correctly on the EC2 instance:

```
# Check all resources in the mern-app namespace
docker exec jenkins kubectl get all -n mern-app

# You should see output similar to:
# NAME                          READY   STATUS    RESTARTS   AGE
# pod/client-xxx                1/1     Running   0          2m
# pod/server-xxx                1/1     Running   0          2m
# pod/mongo-0                   1/1     Running   0          3m
#
# NAME                     TYPE        CLUSTER-IP   PORT(S)
# service/client-service   ClusterIP   10.x.x.x     80/TCP
# service/server-service   ClusterIP   10.x.x.x     5000/TCP
# service/mongo-service    ClusterIP   None         27017/TCP
#
# NAME                     READY   UP-TO-DATE   AVAILABLE
# deployment.apps/client   2/2     2            2
# deployment.apps/server   2/2     2            2
```

Also verify the Ingress is created and has an address:

```
docker exec jenkins kubectl get ingress -n mern-app

# Expected output:
# NAME           CLASS   HOSTS   ADDRESS        PORTS
# mern-ingress   nginx   *       <minikube-ip>  80
```

STEP 13 Forward Traffic from EC2 Public IP to Minikube

Minikube's nginx Ingress listens on the Minikube internal IP, not the EC2 public IP. Use socat to bridge port 80 of the EC2 instance to Minikube:

```
# Start socat in the background — forwards EC2 port 80 to Minikube's port 80
sudo nohup socat TCP-LISTEN:80,fork TCP:$(minikube ip):80 > /tmp/socat.log 2>&1 &
```

Verify the Minikube IP and confirm socat is running:

```
# Print the Minikube cluster IP
minikube ip

# Confirm socat is listening on port 80
sudo netstat -tlnp | grep ':80'

# Check socat log for errors
cat /tmp/socat.log
```

PERSISTENCE	The socat command above runs in the background but will not survive a reboot. To make it persistent, add it to /etc/rc.local or create a systemd service unit.

STEP 14 Access the Application via EC2 Public IP

With socat forwarding port 80 to Minikube's Ingress, the full MERN application is now accessible from any browser using the EC2 instance's public IP address:

Open in browser:

```
http://<EC2_PUBLIC_IP>
```

URL Path	What You Should See
`http://<EC2_PUBLIC_IP>/`	React frontend served by nginx (routed by Ingress to client-service:80)
`http://<EC2_PUBLIC_IP>/api/*`	API endpoints served by the Node.js backend (routed to server-service:5000)

Troubleshooting

Problem	Solution
Cannot connect to EC2 on port 8080	Check EC2 Security Group inbound rules - ensure port 8080 is open for your IP
docker: permission denied	Run: sudo usermod -aG docker $USER && newgrp docker
Minikube fails to start	Ensure at least 2 CPUs and 2 GB free RAM: free -h and nproc
Jenkins cannot push to Docker Hub	Verify dockerhub-credentials ID is correct and Docker Hub password/token is valid
kubectl: connection refused in Jenkins	Ensure jenkins container is on the minikube network: docker network connect minikube jenkins
App not accessible on port 80	Confirm socat is running: sudo netstat -tlnp | grep ':80' and check /tmp/socat.log
Pods in CrashLoopBackOff	Check logs: kubectl logs <pod-name> -n mern-app and verify secrets.yaml values
Ingress has no ADDRESS	Wait for ingress-nginx-controller pod to be Running: kubectl get pods -n ingress-nginx